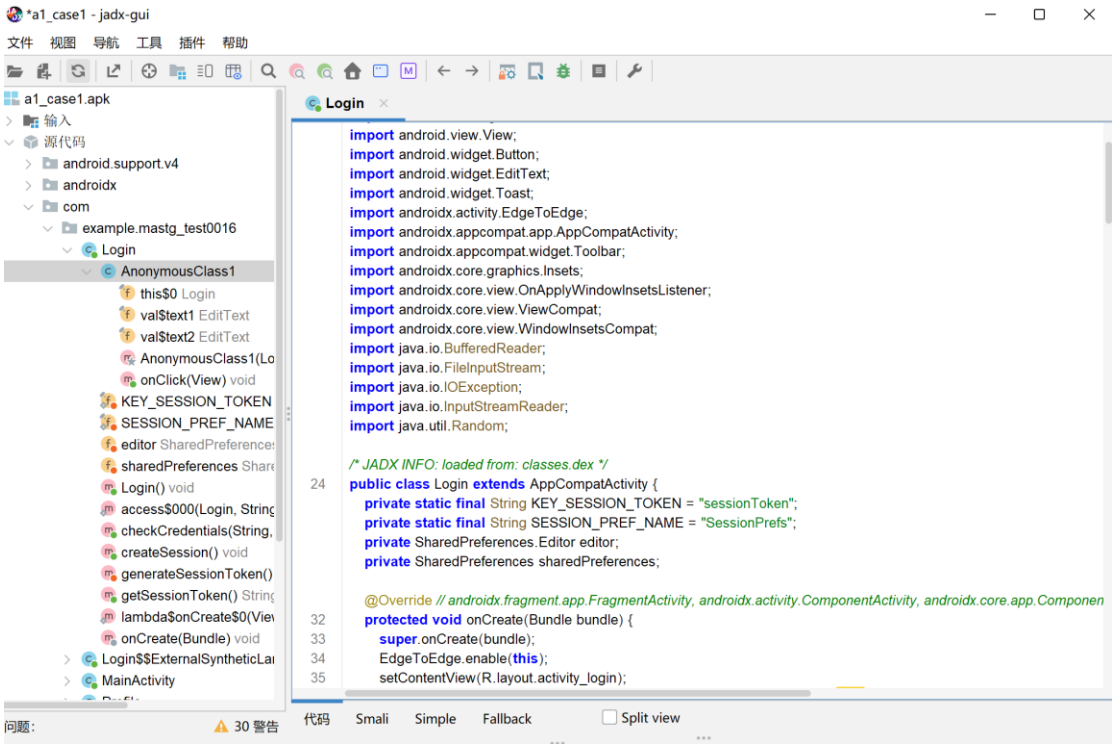
Task1:

APK = Android application package



Task2 understand app

1. App Purpose and Main Features

The application implements a simple local authentication system. Users can register by entering a username and password, which are stored in a local file. During login, the app reads the stored credentials and verifies them against user input. If the credentials match, a session token is generated and saved locally to represent the authenticated state. The user is then allowed to access the profile page and can log out to clear the session.

---

2. System Model

The system consists of several core components:

- **User Interface (UI):** Handles user input for registration and login (MainActivity).
- **Local Storage:** Stores user credentials in a file (credentials.txt).
- **Login Logic:** Reads stored credentials, validates input, and generates a session

token.

- **Session Storage:** Stores the session token in SharedPreferences.
- **Profile Page:** Accessible after authentication and allows logout functionality.

Data Flow:

1. The user enters a username and password in the UI.
2. During registration, credentials are written to local storage.
3. During login, credentials are read from the file and compared with user input.
4. If valid, a session token is generated and stored in SharedPreferences.
5. The app uses this token to grant access to the profile page.
6. On logout, the session token is removed.

Key Assets:

- Username and password
- Local credential file (credentials.txt)
- Session token
- Authentication state

---

3. Assumptions and Attacker Goals

Assumptions:

- The application runs locally without a backend server.
- All authentication and session management are handled on the client side.
- Sensitive data is stored in local storage on the device.

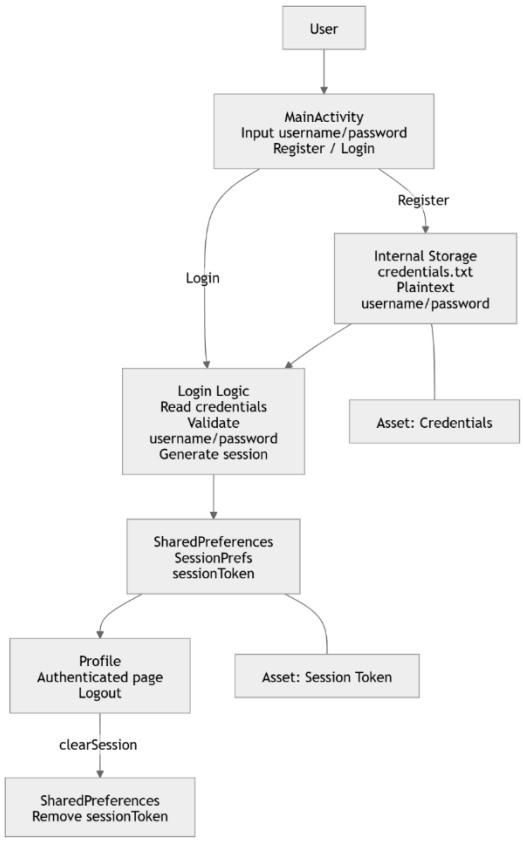
Attacker Goals:

- Obtain user credentials from local storage
- Bypass authentication and access the profile page
- Manipulate or forge session tokens to impersonate a user

Attacker Capabilities:

- Observe and decompile the APK to understand app logic

- Read or modify local files and SharedPreferences
- Interact with the app and provide arbitrary input



Task3 Find Vulnerabilities

1. Identification of Randomness Usage

We performed static analysis on the decompiled source code using JADX and searched for values related to randomness, tokens, and identifiers (e.g., “random”, “token”, “code”).

The following relevant usage was identified:

Class	Method	API Used	Security Role
Login.java	generateSessionToken()	java.util.Random	Session Token

The method generateSessionToken() generates a 16-character token using uppercase letters, lowercase letters, and digits.

---

2. Security Assessment of the Random Generator

The session token is generated using java.util.Random, which is not a cryptographically secure random number generator.

- Random produces predictable outputs based on an internal seed
- It is suitable for non-security purposes (e.g., UI randomness)
- It is not suitable for security-sensitive values such as authentication tokens

In contrast, secure tokens should be generated using APIs such as java.security.SecureRandom.

---

3. Identified Vulnerability

The use of java.util.Random for session token generation is a security-relevant weakness.

A session token serves as an authentication credential. If the randomness used to generate it is predictable, an attacker may be able to:

- Predict or narrow down possible token values
- Reconstruct valid tokens based on observed outputs
- Forge session tokens and impersonate authenticated users

Task4 attack model

What went wrong

The main issue is how the app generates the session token after login. In Login.java, the method generateSessionToken() uses java.util.Random to create a 16-character token.

This is a problem because Random is not designed for security. Its output is predictable compared to a secure generator like SecureRandom. However, a session token should be unpredictable, since it represents a logged-in user's identity. If the token can be guessed, the authentication mechanism becomes weak.

Why this is a security problem

A session token is essentially a secret that proves a user is logged in. If the token is generated using a weak random source, its randomness is limited, and attackers may be able to predict or reproduce valid tokens.

In this app, the risk is even higher because the token is stored locally in SharedPreferences, and there is no additional protection or verification. This makes the session mechanism rely entirely on a predictable value.

Attacker knowledge and capabilities

This attack is realistic because the attacker can:

- Decompile the APK and see that Random is used
- Understand the token format (16 characters, letters and digits)
- Read or modify local data such as SharedPreferences
- Observe when login happens or extract existing tokens

All of this is feasible on a rooted device or in a debugging environment.

Attack scenario

1. The attacker decompiles the APK and finds the token generation logic.
2. They confirm that java.util.Random is used.
3. The attacker observes a generated token or the login timing.
4. Using this information, they reduce the possible token space or predict future tokens.
5. The attacker injects a forged or predicted sessionToken into SharedPreferences.
6. The app accepts the token and treats the attacker as a logged-in user